

premise_correction_20260822

BOB-168 — the premise is FALSE: the entry does not dangle

Revision: 1 **Last modified:** 2026-08-22T10:30:12Z **Evidence class:** RUNTIME + git history (§11.4.226) — filesystem probes and git log on the real repository, with control needles. **Mandate:** §11.4.124 (investigate before removing), §11.4.201(7)(9) (field identity; a null is not evidence until the instrument is proven to see), §11.4.6 (no guessing).

The finding

scaling_horizontal_challenge.sh **exists, is executable, and has never been deleted.** So do all fourteen other entries, and the meta-runner. There is nothing to remove and nothing to author. Acceptance (b) as filed is void.

```
submodules/challenges/challenges/scripts/  
scaling_horizontal_challenge.sh  
-f TRUE -x TRUE  
size 3107 mode 755  
sha256  
d7f1db44f515bcc6e7feb390cf8591afafafbb8b7a1ed68124bfdc6d0447da79
```

Full roster probe — all 16 named entries, -f and -x both true:

entry	-f	-x
no_suspend_calls_challenge.sh	yes	yes
host_no_auto_suspend_challenge.sh	yes	yes
bluff_scanner_challenge.sh	yes	yes
anchor_manifest_challenge.sh	yes	yes
challenges_compile_challenge.sh	yes	yes

entry	-f	-x
challenges_functionality_challenge.sh	yes	yes
challenges_unit_challenge.sh	yes	yes
chaos_failure_injection_challenge.sh	yes	yes
ddos_health_flood_challenge.sh	yes	yes
mutation_ratchet_challenge.sh	yes	yes
recording_pipeline_challenge.sh	yes	yes
scaling_horizontal_challenge.sh	yes	yes
stress_sustained_load_challenge.sh	yes	yes
ui_terminal_interaction_challenge.sh	yes	yes
ux_end_to_end_flow_challenge.sh	yes	yes
challenges_describe_challenge.sh (meta)	yes	yes

Control needle (§11.4.201(7)(b)). The same -f probe, same path, same shell, against `definitely_absent_needle_challenge.sh` returns FALSE. The instrument can see absence, so the sixteen TRUEs are evidence and not a blind instrument’s uniform answer.

Where the mis-measurement came from

The runner does not read from `challenges/`. Line 50:

```
CHALLENGES_DIR="${PROJECT_ROOT}/submodules/challenges/challenges/
scripts"
```

The filing commit c229b11 recorded:

```
scripts/run_all_challenges.sh:66 lists
scaling_horizontal_challenge.sh; challenges/scripts/ has no such
file. Verified by invocation, not by reading the report.
```

Both halves of that sentence are true in isolation and wrong together. The repo has two similarly-named directories and two different runners:

	challenges/scripts/	submodules/ challenges/ challenges/scripts/
read by	challenges/scripts/ run_all_challenges.sh	scripts/ run_all_challenges.sh
selection	glob *_challenge.sh	hardcoded list

	challenges/scripts/	submodules/ challenges/ challenges/scripts/
can dangle?	no — enumerates what exists	yes — by construction
holds the file?	no	yes

The check was run against the directory the *other* runner uses. This is a §11.4.201(9) field-identity error: the name matched, the referent did not. It is also why “verified by invocation” overstates what happened — the aggregator was never invoked; a directory listing was.

The irony is worth recording because it is the lesson: **the very same commit correctly caught a sibling agent making this class of error on BOB-167** (a zero-hit from searching one symbol name while the wiring used another), applied a control needle there, and then omitted the same needle one paragraph later. A control needle proves the instrument can see; it does not prove the instrument is pointed at the thing under test. Those are two different checks and this round needed both.

§11.4.124 git history — never dropped, deliberately added

```
# parent repo – the entry's introduction
1c959ba feat: complete 6-task session – HelixQA banks,
Challenges/Containers
    subdeps, workable-items SQLite, export-sync expansion,
boba-ctl CLI
    (introduces scripts/run_all_challenges.sh; line 37 = the
entry)
```

```
# parent repo – deletions of that filename, anywhere, any branch
git log --diff-filter=D --all --name-only | grep
scaling_horizontal
-> (empty)
```

```
# submodule – the challenge's introduction
873c0b1 feat(challenges): CONST-050(B) submodule cascade – 6 new
anti-bluff
    test types
```

```
# submodule – deletions
git log --all --diff-filter=D --name-only | grep
```

```
scaling_horizontal  
-> (empty)
```

Pickaxe control needle: `git log -S run_all_challenges.sh --all` returns a non-empty, plausible set, so the empty deletion results are real absences and not a broken query.

Verdict: the entry and the challenge were added in the same era as part of a deliberate six-test-type cascade, and neither has ever been deleted. The “silently-dropped challenge” possibility the filing flagged as more serious is **ruled out by evidence**, not assumed away.

§11.4.233(G) — the pointer is fetchable

The challenges submodule pointer 072724af is present on `github/main` and `gitlab/main`, and the object resolves locally. Not an ABSENT/UNFETCHABLE dependency-pointer defect.

What survives

The roster is clean; the **exit semantics are not**. That defect is real, reachable, and worse than the one filed — see `runtime_confirmation_20260822.md` and `decision_missing_vs_skip.md`.